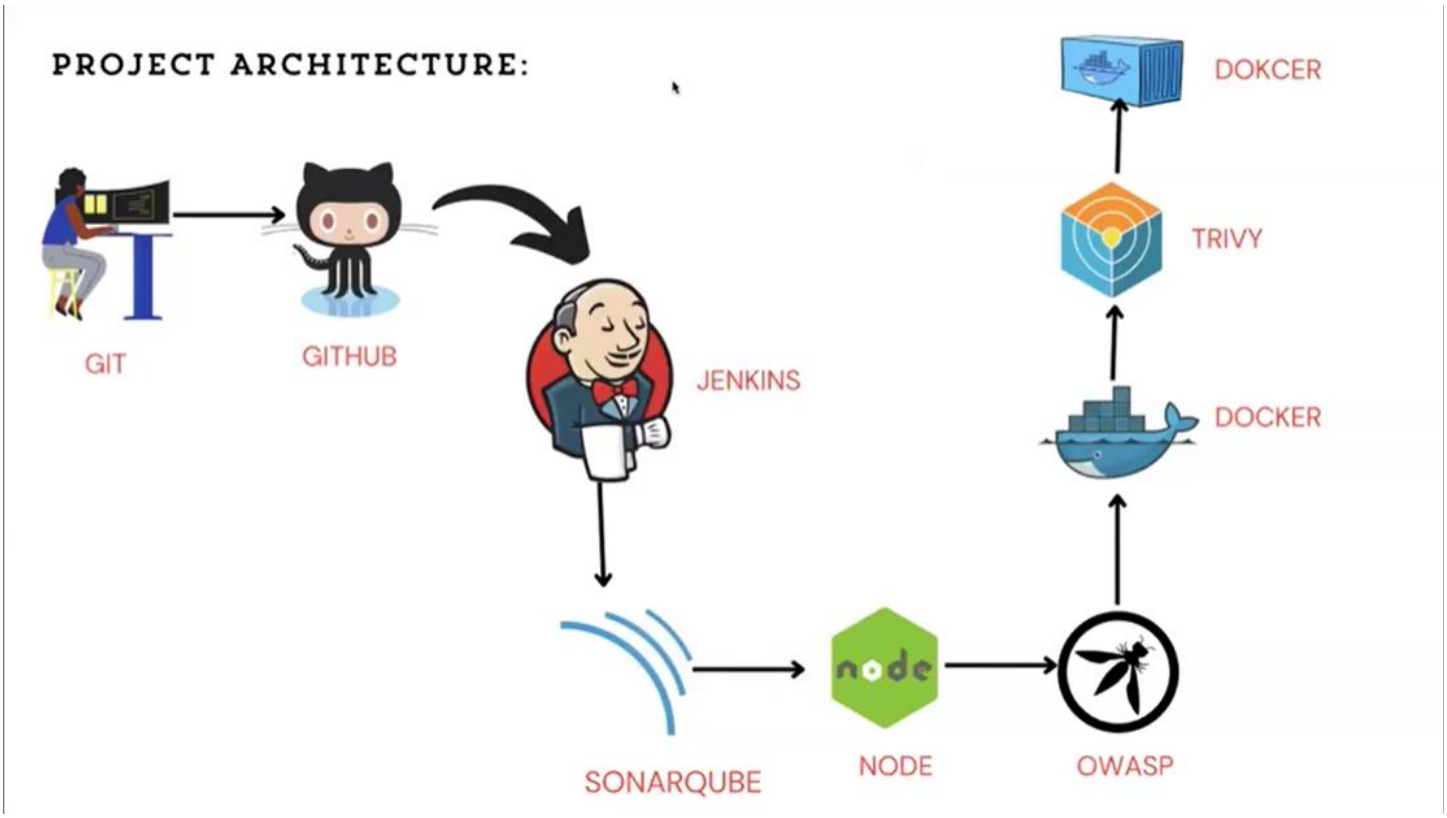
PROJECT

IMPLEMENTING A CI/CD PIPELINE WITH INTEGRATED SECURITY AND QUALITY CHECKS



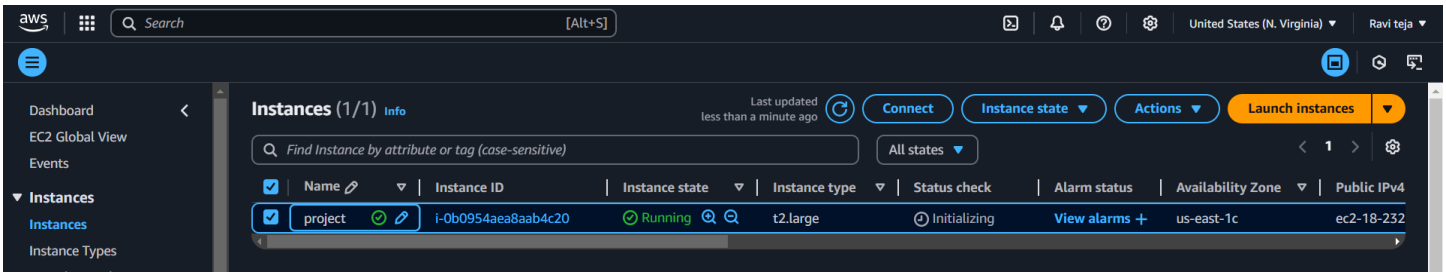
TRIVY :

TRIVY IS AN OPEN-SOURCE SECURITY SCANNER DEVELOPED BY AQUA SECURITY. IT CHECKS FOR VULNERABILITIES IN CONTAINER IMAGES, FILESYSTEMS, GIT REPOSITORIES, VIRTUAL MACHINES, AND KUBERNETES CONFIGURATIONS2. TRIVY SCANS FOR OS PACKAGES, SOFTWARE DEPENDENCIES, KNOWN VULNERABILITIES (CVES), SENSITIVE INFORMATION, SECRETS, AND SOFTWARE LICENSES. IT SUPPORTS MULTIPLE PROGRAMMING LANGUAGES, OPERATING SYSTEMS, AND PLATFORMS2.

OWASP :

THE OWASP (OPEN WEB APPLICATION SECURITY PROJECT) IS A NON-PROFIT FOUNDATION THAT WORKS TO IMPROVE THE SECURITY OF SOFTWARE. IT OFFERS A VARIETY OF TOOLS, RESOURCES, AND GUIDELINES TO HELP DEVELOPERS CREATE SECURE APPLICATIONS. KEY PROJECTS INCLUDE THE OWASP TOP TEN LIST, ZAP (ZED ATTACK PROXY), AND DEPENDENCY-CHECK, WHICH IDENTIFY AND MITIGATE VULNERABILITIES IN WEB APPLICATIONS. OWASP AIMS TO ENHANCE AWARENESS AND COLLABORATION WITHIN THE COMMUNITY TO BUILD MORE SECURE SOFTWARE SOLUTIONS.

STEP-1: LAUNCH AN EC2 INSTANCE WITH T2.LARGE AND MINIMUM OF 20GB STORAGE



STEP-2: CONNECT THE SERVER

1. **CREATE A FILE WITH .SH EXTENSION (jenkins.sh)**
2. **GO TO INSERT MODE BY CLICKING I AND COPY PASTE THE BELOW SCRIPT THEN SAVE AND EXIT THE FILE WITH (click on escape then :wq)**

#STEP-1: INSTALLING GIT
yum install git -y

#STEP-2: GETTING THE REPO (jenkins.io --> download -- > redhat)
sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key

#STEP-3: DOWNLOAD JAVA17 AND JENKINS
yum install java-17-amazon-corretto -y
yum install jenkins –y
#STEP-4: RESTARTING JENKINS (when we download service it will on stopped state)
systemctl start jenkins.service
systemctl status jenkins.service

STEP-3: AFTER COMPLETION OF SCRIPT EXECUTION AND CONNECT TO DASHBOARD COPY THE PUBLIC IP ADDRESS OF THE SERVER AND AND MAKE A PASTE ON THE NEW TAB WITH JENKINS PORT NUMBER (8080) PUBLIC_IP:8080

1. TO UNLOCK THE JENKINS: cat /var/lib/jenkins/secrets/initialAdminPassword
2. INSTALL SUGGESTED PLUGINS ADD USER NAME, PASSWORD AND MAIL ID CONNECT TO DASHBOARD
3. ENTER THE REQUIRED DETAILS AND YOU WILL ENTER TO JENKINS DASHBOARD
4. CLICK ON NEW ITEM THEN SELECT PIPELINE JOB AND CLICK ON OKK

STEP-4: INSTALL DOCKER AND START AND CHECK THE STATUS OF DOCKER

TO INSTALL DOCKER : yum install docker –y

TO START DOCKER : systemctl start docker

TO CHECK THE STATUS OF DOCKER : systemctl status docker

```
[root@ip-172-31-38-74 ~]# yum install docker -y
Loaded plugins: extras_suggestions, langpacks, priorities, update-motd
Existing lock /var/run/yum.pid: another copy is running as pid 5553.
Another app is currently holding the yum lock; waiting for it to exit...
  The other application is: yum
    Memory :  66 M RSS (286 MB VSZ)
    Started: Sat Feb 15 05:32:41 2025 - 00:03 ago
    State  : Running, pid: 5553
Another app is currently holding the yum lock; waiting for it to exit...
  The other application is: yum
    Memory : 206 M RSS (425 MB VSZ)
    Started: Sat Feb 15 05:32:41 2025 - 00:05 ago
    State  : Running, pid: 5553
```

```
[root@ip-172-31-38-74 ~]# systemctl start docker
[root@ip-172-31-38-74 ~]# systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; disabled; vendor preset: disabled)
   Active: active (running) since Sat 2025-02-15 05:33:07 UTC; 7ms ago
     Docs: https://docs.docker.com
  Process: 5965 ExecStartPre=/usr/libexec/docker/docker-setup-runtimes.sh (code=exited, status=0/SUCCESS)
  Process: 5963 ExecStartPre=/bin/mkdir -p /run/docker (code=exited, status=0/SUCCESS)
 Main PID: 5967 (dockerd)
    Tasks: 9
   Memory: 28.9M
   CGroup: /system.slice/docker.service
           └─5967 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock --default-ulimit nofile=3276...
```

STEP-5: INSTALL SONARQUBE USING DOCKER IMAGE AND CHECK THE DOCKER CONTAINER HAS CREATED

TO INSTALL SONARQUBE BY DOCKER IMAGE : docker run –d --name sonar –p 9000:9000 sonarqube:lts-community

TO CHECK SONARDOCKER CONTAINER : docker ps

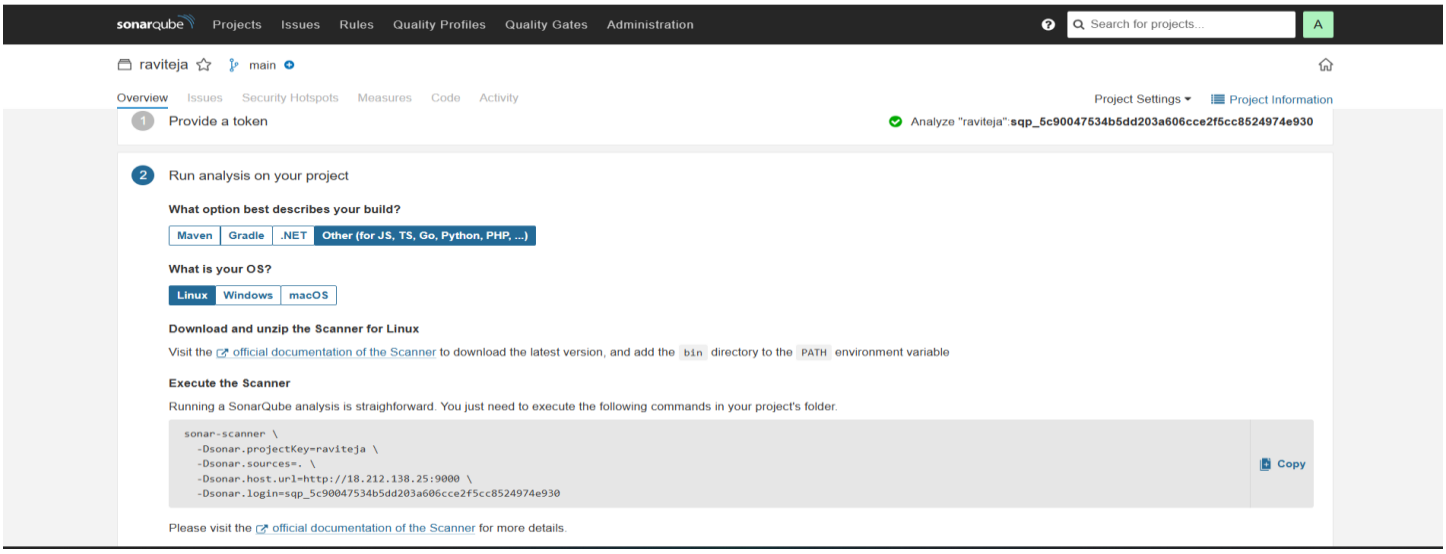
```
[root@ip-172-31-38-74 ~]# docker run -d --name sonar -p 9000:9000 sonarqube:lts-community
Unable to find image 'sonarqube:lts-community' locally
lts-community: Pulling from library/sonarqube
9cb31e2e37ea: Pull complete
13876c96bdc5: Pull complete
25fdfc9faee8: Pull complete
b682cc54ed35: Pull complete
4615ed3a3407: Pull complete
c2e1e3bdd7bc: Pull complete
b2dce0ce5cad: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:95b1826b68e606678882148e28c01c70dc0efe464a0d0c568570b17eedf1a9f9
Status: Downloaded newer image for sonarqube:lts-community
  37fad7412711a11389cfc476e898ad9da3a9f74d741b1302554fdb5d752d1add
[root@ip-172-31-38-74 ~]#
[root@ip-172-31-38-74 ~]# docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED          STATUS          PORTS
                      NAMES
37fad7412711   sonarqube:lts-community   "/opt/sonarqube/dock…"   14 seconds ago   Up 10 seconds   0.0.0.0:9000->9000/tc
p, :::9000->9000/tcp   sonar
[root@ip-172-31-38-74 ~]#
```

TO CONNECT TO DASHBOARD COPY THE PUBLIC IP ADDRESS OF THE SERVER AND AND MAKE A PASTE ON THE NEW TAB WITH SONARQUBE PORT NUMBER (9000) PUBLIC_IP:9000

1. AFTER GOING TO DASHBOARD IT WILL ASK USERNAME AND PASSWORD AND USERNAME AND PASSWORD

WILL BE ADMIN

2. UPDATE THE PASSWORD AND IN DASHBOARD CLICK ON VREATE PROJECT
3. GIVE A NAME TO PROJECT AND CLICK ON SETUP
4. CLICK ON LOCALLY THEN CLICK ON GENERATE
5. SELECT OTHER (FOR JS, TS, GO, PYTHON)
6. SELECT LINUX
7. YOU CAN SEE SOME GENERATED SCRIPT



STEP-6: INSTALL TRIVY

1. **wget https://github.com/aquasecurity/trivy/releases/download/v0.18.3/trivy_0.18.3_Linux64bit.tar.gz**
2. **tar zxvf trivy_0.18.3_Linux-64bit.tar.gz**
3. **sudo mv trivy /usr/local/bin/**
4. **vim .bashrc (open vim .bashrc go to last line paste this line export PATH=$PATH:/usr/local/bin/)**
5. **export PATH=$PATH:/usr/local/bin/**
6. **source .bashrc**
7. **CHECK THE VERSION WHETHER TRIVY IS INSTALLED OR NOT**

```
[root@ip-172-31-38-74 ~]#
[root@ip-172-31-38-74 ~]# wget https://github.com/aquasecurity/trivy/releases/download/v0.18.3/trivy_0.18.3_Linux-64bit.
tar.gz
--2025-02-15 05:33:07--  https://github.com/aquasecurity/trivy/releases/download/v0.18.3/trivy_0.18.3_Linux-64bit.tar.gz
Resolving github.com (github.com)... 140.82.113.3
Connecting to github.com (github.com)|140.82.113.3|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://objects.githubusercontent.com/github-production-release-asset-2e65be/180687624/72009f80-bc6e-11eb-9322
-484e3de7e75d?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=releaseassetproduction%2F20250215%2Fus-east-1%2Fs3%2Faws
4_request&X-Amz-Date=20250215T053307Z&X-Amz-Expires=300&X-Amz-Signature=6fb84d3809c1c731914d5912b1a496bfaf350d0ce8e3ef6a
10d1f6aa1f6a1205&X-Amz-SignedHeaders=host&response-content-disposition=attachment%3B%20filename%3Dtrivy_0.18.3_Linux-64b
it.tar.gz&response-content-type=application%2Foctet-stream [following]
--2025-02-15 05:33:07--  https://objects.githubusercontent.com/github-production-release-asset-2e65be/180687624/72009f80
-bc6e-11eb-9322-484e3de7e75d?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=releaseassetproduction%2F20250215%2Fus-ea
st-1%2Fs3%2Faws4_request&X-Amz-Date=20250215T053307Z&X-Amz-Expires=300&X-Amz-Signature=6fb84d3809c1c731914d5912b1a496bfa
f350d0ce8e3ef6a10d1f6aa1f6a1205&X-Amz-SignedHeaders=host&response-content-disposition=attachment%3B%20filename%3Dtrivy_0
.18.3_Linux-64bit.tar.gz&response-content-type=application%2Foctet-stream
Resolving objects.githubusercontent.com (objects.githubusercontent.com)... 185.199.109.133, 185.199.110.133, 185.199.111
.133, ...
```

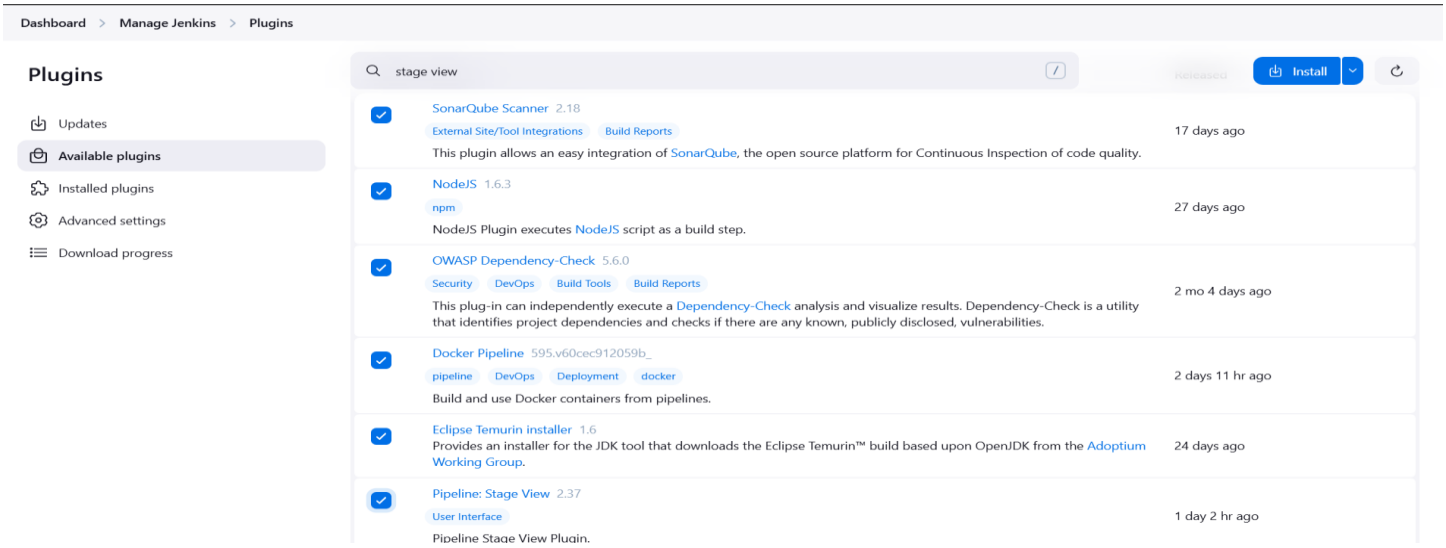
```
[root@ip-172-31-38-74 ~]# tar zxvf trivy_0.18.3_Linux-64bit.tar.gz
LICENSE
README.md
contrib/asff.tpl
contrib/gitlab.tpl
contrib/html.tpl
contrib/junit.tpl
contrib/sarif.tpl
trivy
```

```
[root@ip-172-31-38-74 ~]# sudo mv trivy /usr/local/bin/
[root@ip-172-31-38-74 ~]# vim .bashrc
[root@ip-172-31-38-74 ~]#
[root@ip-172-31-38-74 ~]# source .bashrc
[root@ip-172-31-38-74 ~]#
[root@ip-172-31-38-74 ~]# trivy --version
Version: 0.18.3
```

STEP-7: OPEN JENKINS→ MANAGE JENKINS→ PLUGINS→ AVAILABE PLUGINS

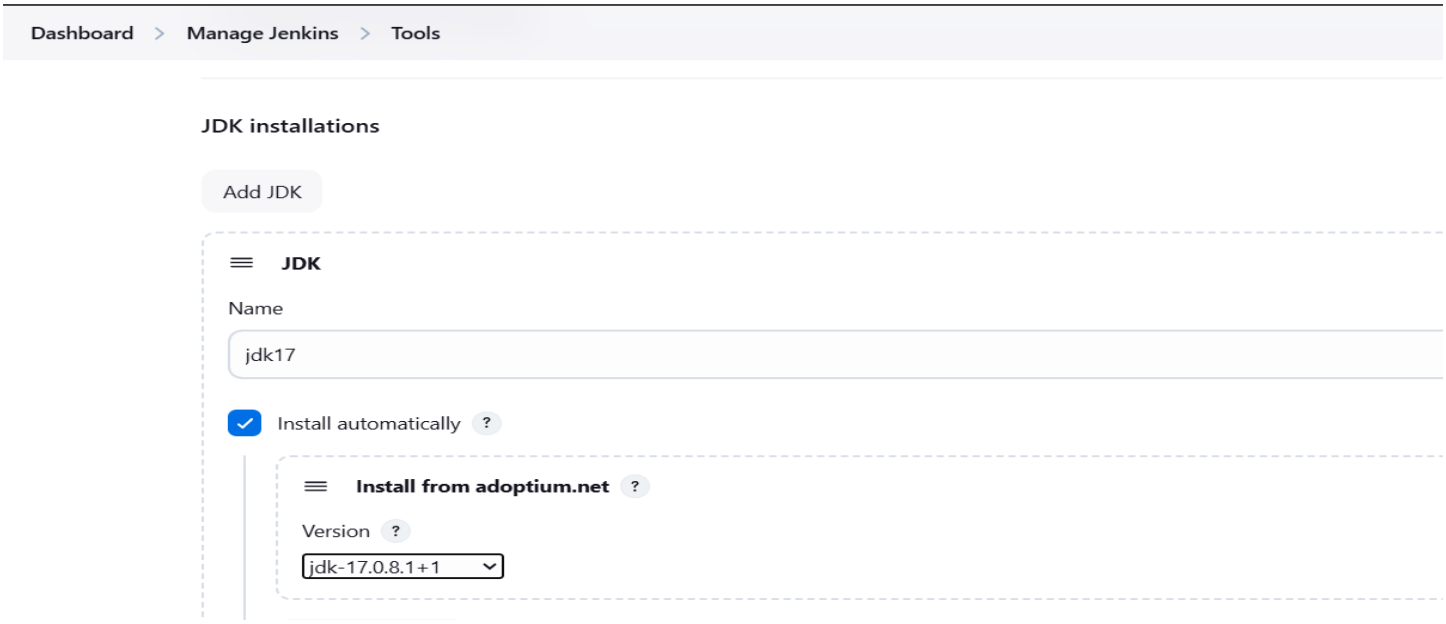
1. **SONARQUBE SCANNER**
2. **NODEJS 16**
3. **OWASP DEPENDENCY CHECK**
4. **DOCKER PIPELINE**
5. **ECLIPSE**
6. **PIPELINE STAGE VIEW**

THESE ARE THE REQUIRED PLUGINS TO INSTALL AND THEN CLICK ON INSTALL



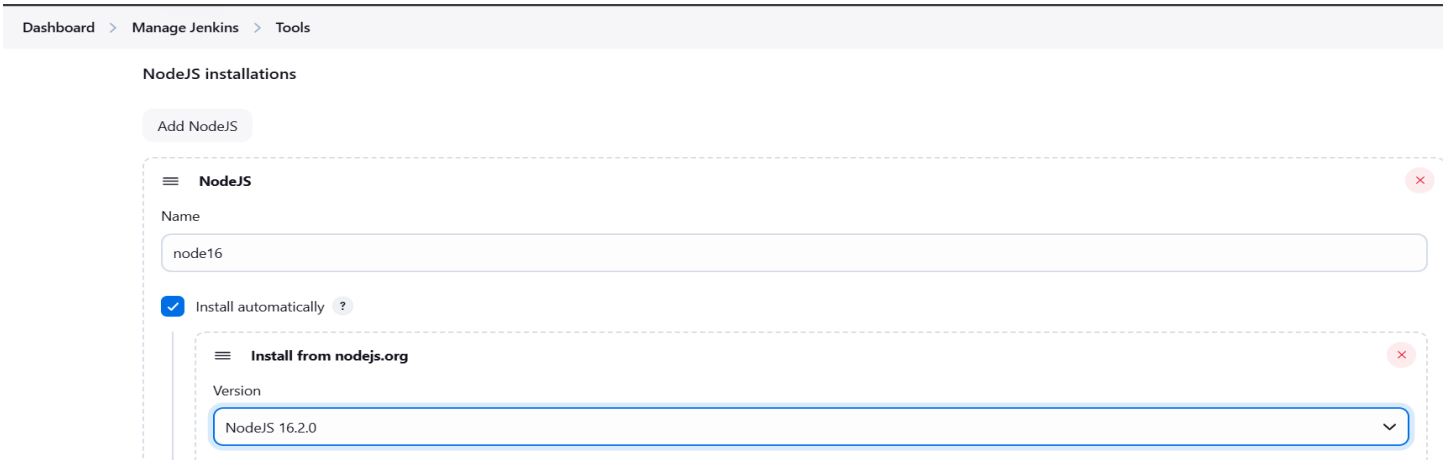
STEP-8: OPEN MANAGE JENKINS→ TOOLS → GO TO JDK INSTALLATIONS

1. **CLICK ON ADD JDK**
2. **GIVE JDK NAME**
3. **CLICK ON INSTALL AUTOMATIALLY**
4. **SELECT INSTALL FROM ODOPTIUM .NET**
5. **SEARCH FOR VERSION 17.0.8.1+1**



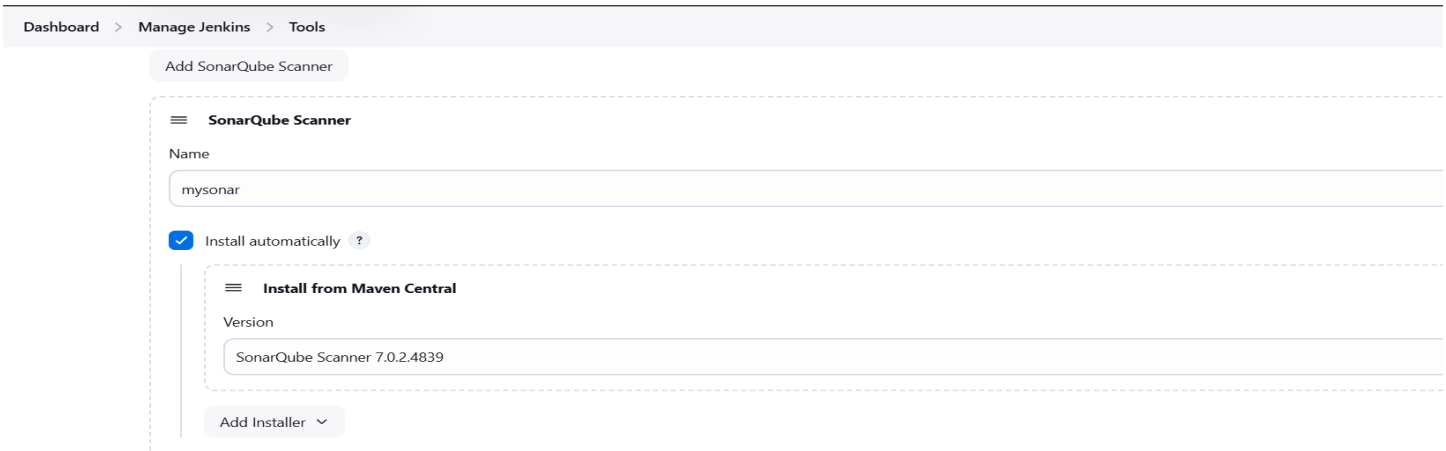
STEP-9: OPEN MANAGE JENKINS→ TOOLS → GO TO NODEJS INSTALLATIONS

1. **CLICK ON ADD NODEJS**
2. **GIVE THE NAME TO NODEJS**
3. **SLECT THE VERSION 16.2.0**



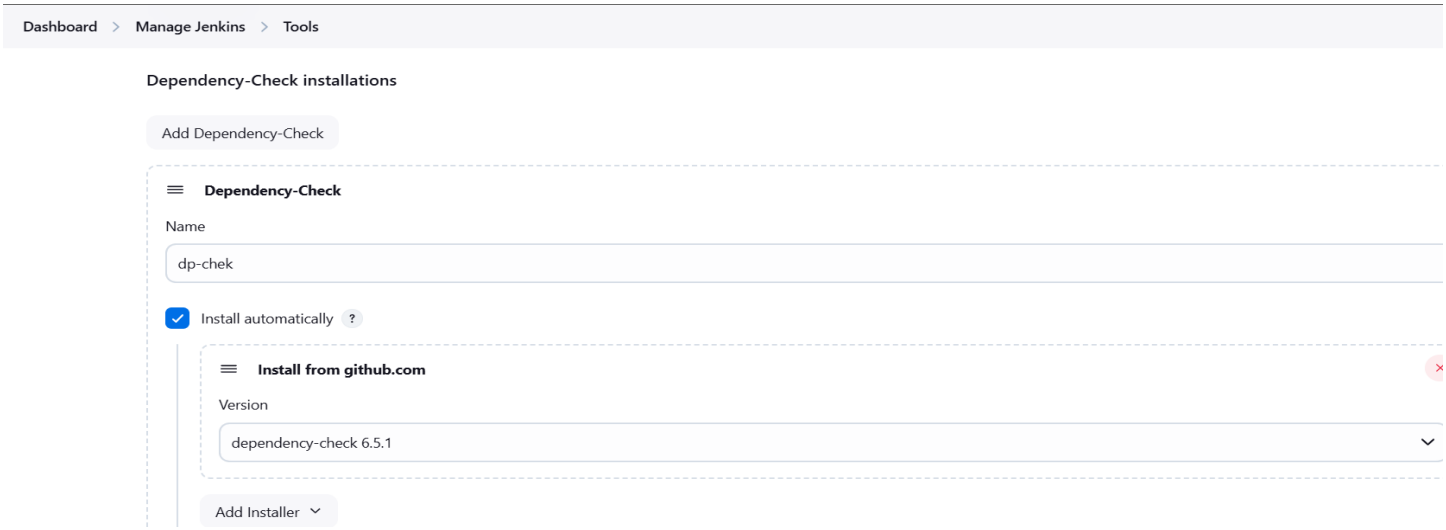
STEP-10: OPEN MANAGE JENKINS→ TOOLS → GO TO SONARQUBE SCANNER INSTALLATIONS

1. **GIVE THE NAME TO THE SONAR**
2. **CLICK ON INSTALL AUTOMATICALLY**
3. **SELECT THE LATEST VERSION**



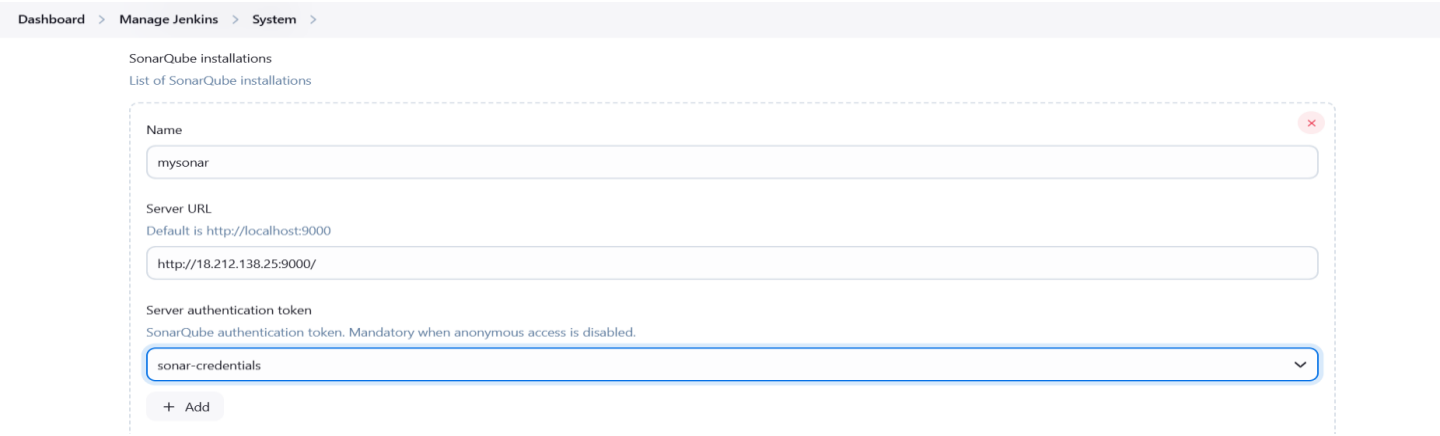
STEP-11: OPEN MANAGE JENKINS→ TOOLS → GO TO DEPENDENCY-CHECK INSTALLATIONS

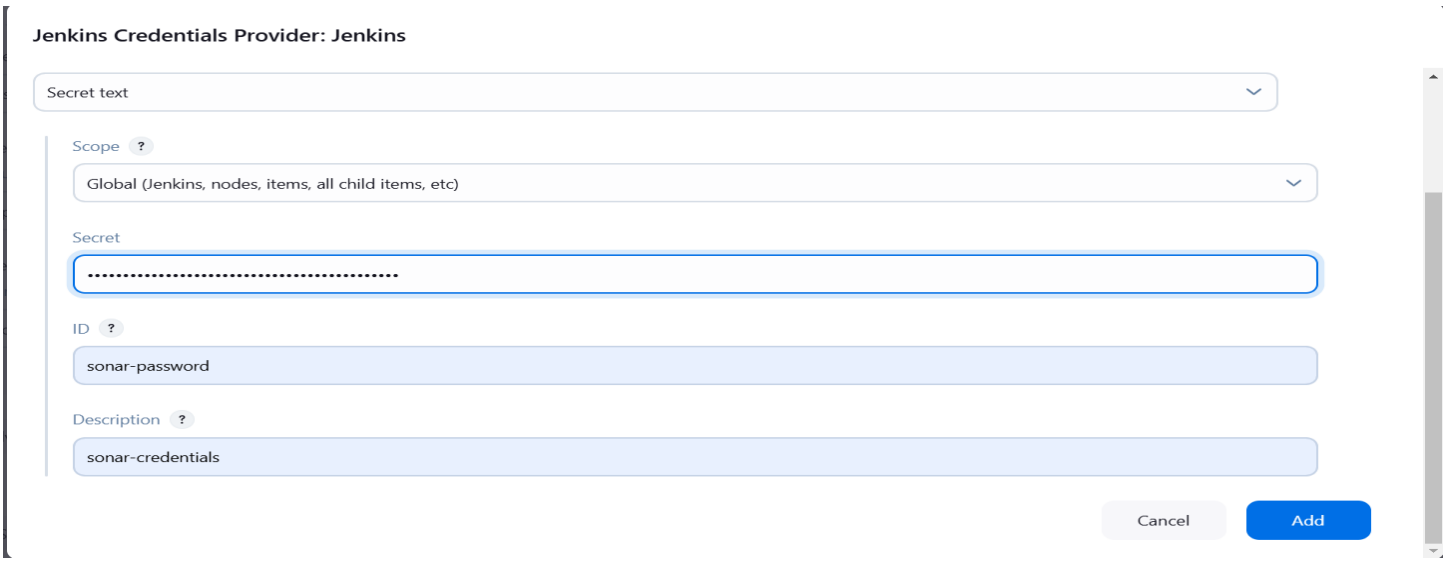
1. **CLICK ON ADD DEPENDENCY CHECK**
2. **GIVE THE NAME TO THE DEPENDENCY CHECK**
3. **SELECT THE VERSION 6.5.1 AND THEN CLICK ON SAVE**



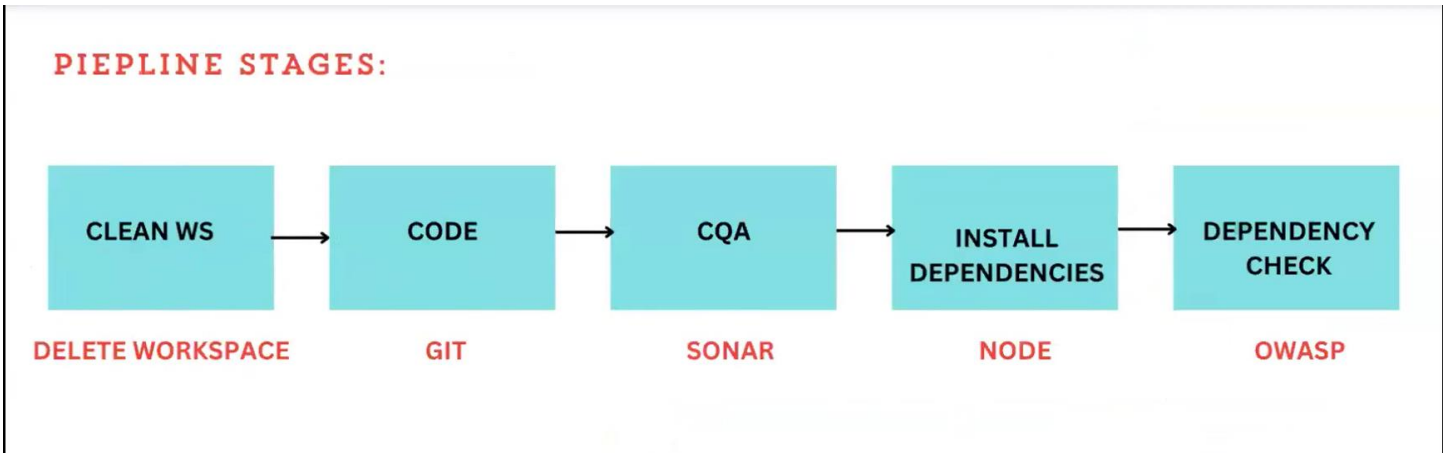
STEP-12: OPEN MANAGE JENKINS→ GO TO SYSTEM

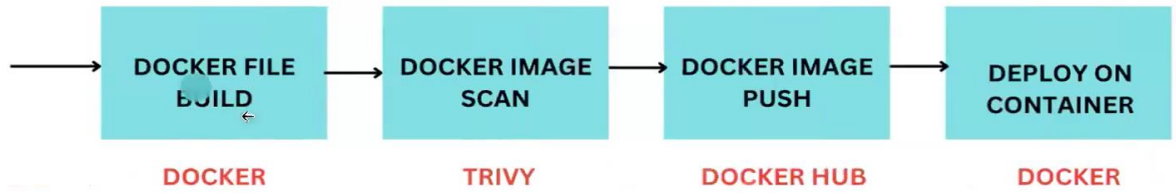
1. **GO TO SONARQUBE INSTALLATIONS**
2. **GIVE THE NAME AND GIVE THE SONARQUBE URL AND THEN CLICK ON ADD FOR SERVER AUTHENTICATION TOKEN**
3. **SELECT THE SECRET TEXT FROM KIND GO TO SONARQUBE SERVER DASHNOARD AND GO TO MY ACCOUNT → GO TO SECURITY AND GIVE A NAME TO TOKEN THEN CLICK ON GENERATE TOKENS AND COPY THE TOKEN**
4. **THEN GO TO JENKINS SERVER PASTE IT ON THE SECRET AND GIVE THE ID AND DESCRIPTION TO THAT AND CLICK ON ADD THEN SELECT SONAR CREDENTIALS IN SERVER AUTHENTICATION TOKEN**





PIPELINE STAGES :





STEP-13: GO TO DASHBOARD → GO TO PIPELINE JOB → GO TO CONFIGURE → WRITE PIPELINE TO GET THE CODE FROM GITHUB INTEGRATE TOOLS AND SONAR AND BEFORE RUNNING THE PIPELINE RUN THE COMMAND

(chmod 777 ///var/run/docker.sock)

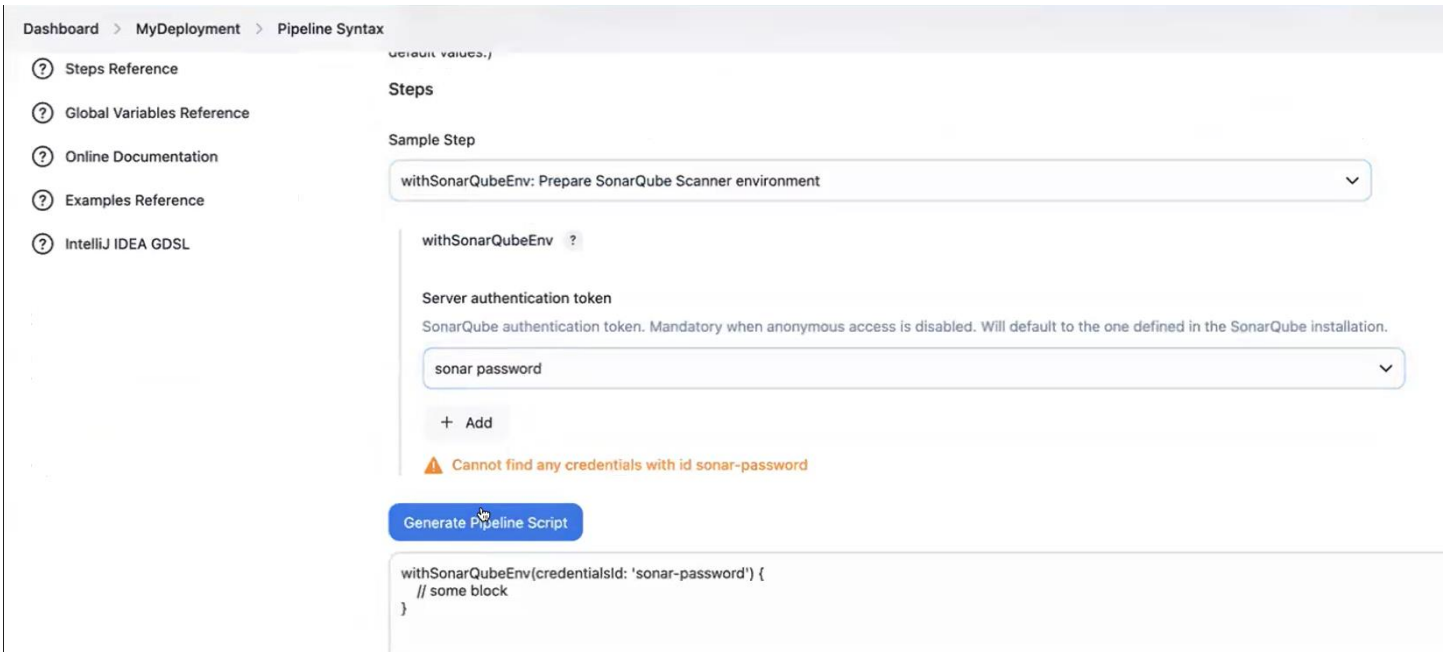
```
pipeline {
  agent any
  tools {
    jdk 'jdk17'
    nodejs 'node16'
  }
  environment {
    SCANNER_HOME=tool 'mysonar'
  }
  stages {
    stage ("Clean") {
      steps {
        cleanWs()
      }
    }
    stage ("Code") {
      steps {
        git 'https://github.com/RAVITEJA3929/Zomato-Project.git'
      }
    }
  }
}
```

SAVE THE BUILD THE JOB

STEP-14: GO TO PIPELINE SYNTAX

1. **SELECT (withSonarQubeEnv)**
2. **SELECT SONARQUBE CREDENTIALS AND THEN CLICK ON GENERATE PIPELINE SCRIPT BUTTON**
3. **COPY THE SCRIPT THEN PASTE IT IN PIPELINE SCRIPT**

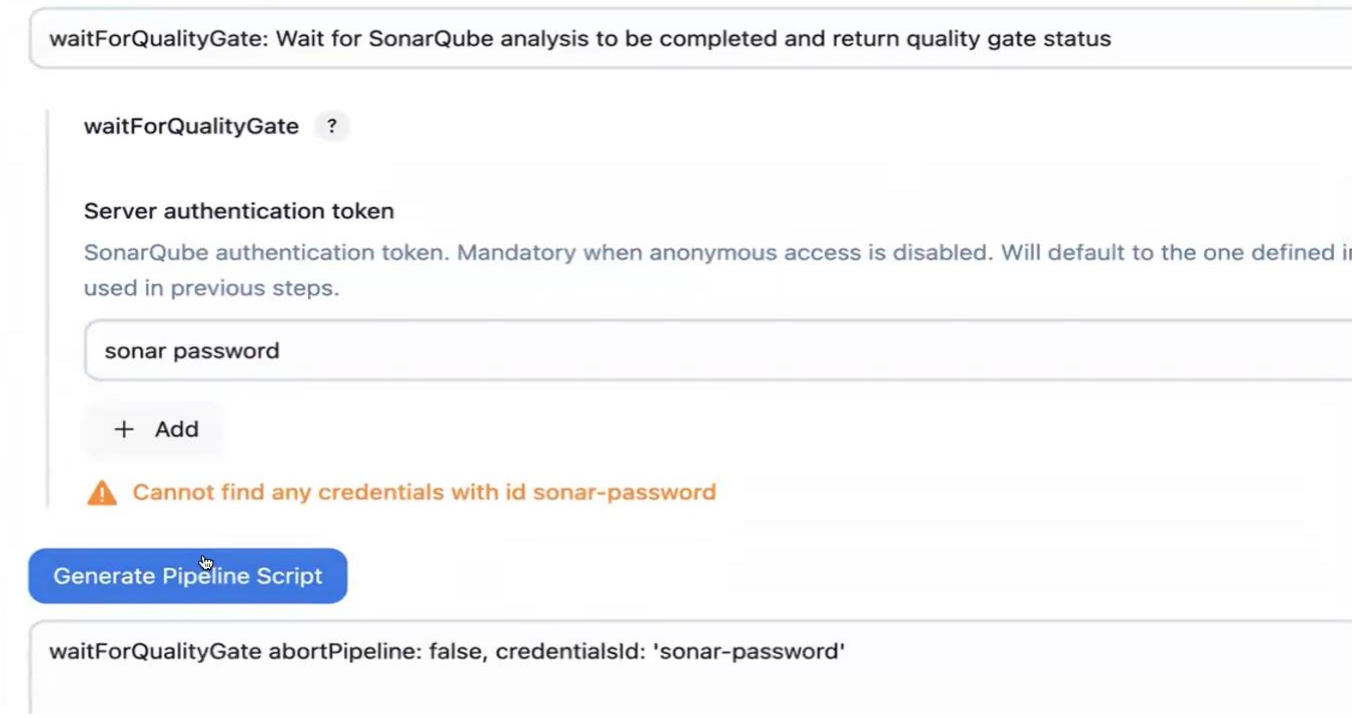
```
STAGE("SONARQUBE ANALYSIS") {
        STEPS{
          WITHSONARQUBEENV('MYSONAR') {
            SH ''' $SCANNER_HOME/BIN/SONAR-SCANNER -DSONAR.PROJECTNAME=ZOMATO \
            -DSONAR.PROJECTKEY=ZOMATO '''
          }
        }
      }
```



STEP-15: GO TO PIPELINE SYNTAX

1. **SELECT (waitForQualityGate)**
2. **SELECT SONARQUBE CREDENTIALS AND THEN CLICK ON GENERATE PIPELINE SCRIPT BUTTON**
3. **COPY THE SCRIPT THEN PASTE IT IN PIPELINE SCRIPT**
4. **SAVE AND BUILD THE JOB**

```
STAGE ("QUALITY GATES") {
        STEPS {
          SCRIPT {
            WAITFORQUALITYGATE ABORTPIPELINE: FALSE, CREDENTIALSID: 'SONAR-TOKEN'
          }
        }
      }
```



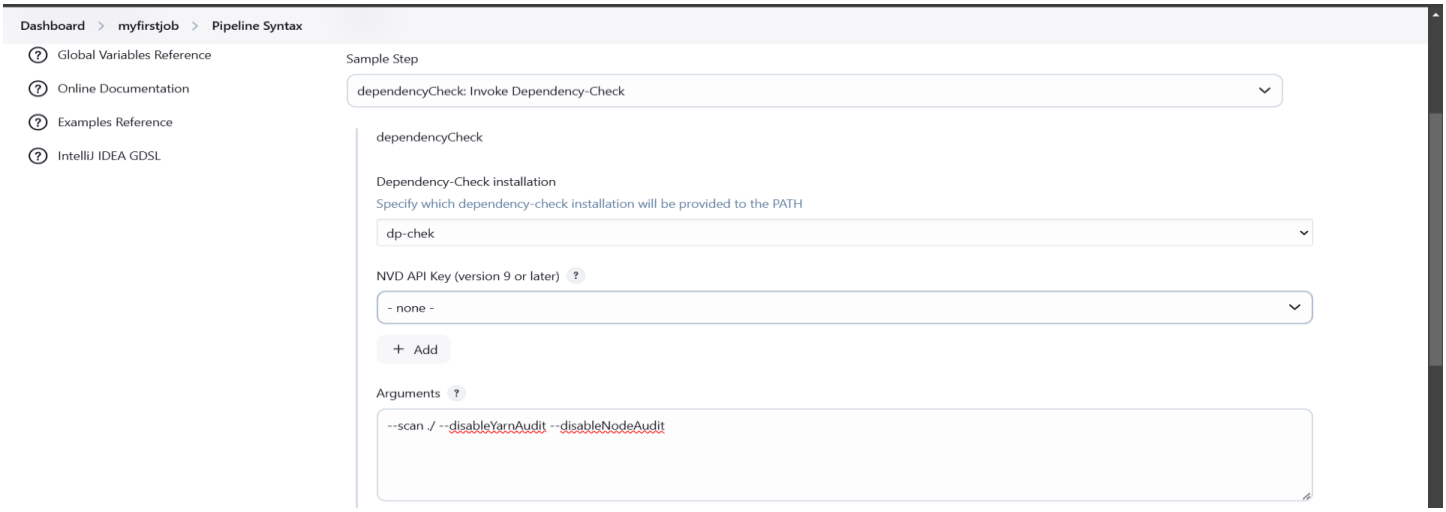
STEP-16: BEFORE GOING TO OWASP WE INSTALL NODEJS AND THEIR DEPENDENCIES BY " npm install"

```
stage ("Install dependencies") {
        steps {
          sh 'npm install'
        }
    }
```

STEP-17: GO TO PIPELINE SYNTAX

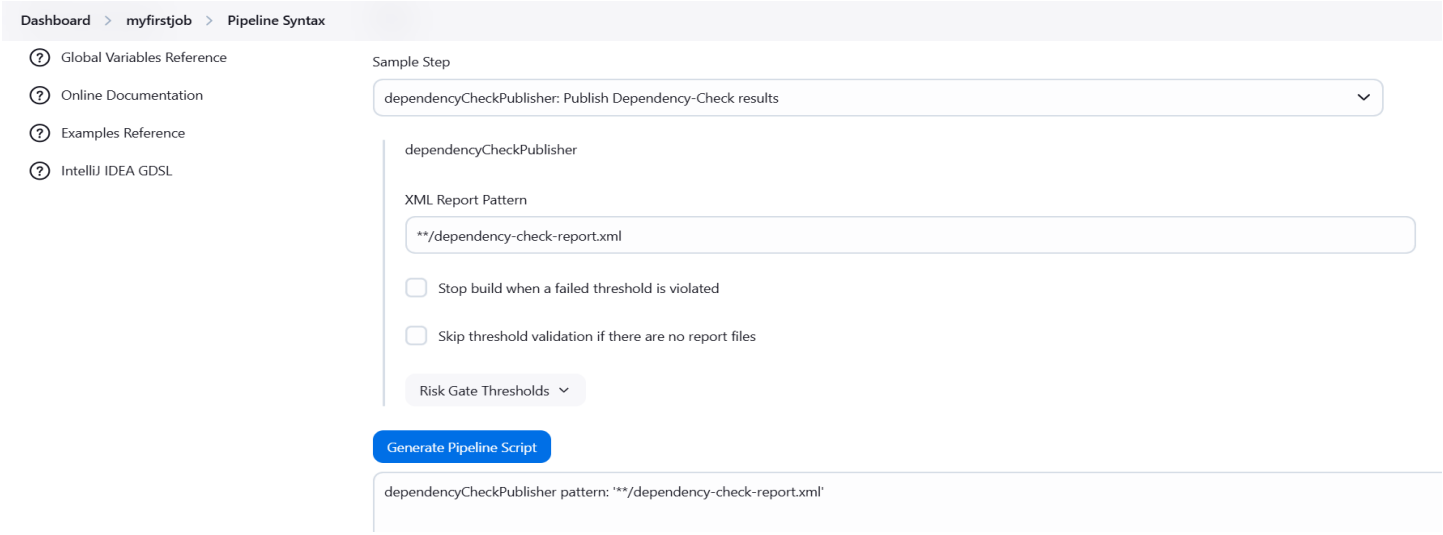
1. **IN THIS STEP WE USE OWASP TO CHECK WHETHER DEPENDENCIES ARE INSTALLED OR NOT**
2. **SELECT (dependencucheck invoke dependency-check)**
3. **PASS THE ARGUMENTS IN THE ARGUMENT BOX (--scan ./ --disableYarnAudit --disableNodeAudit)**
4. **COPY THE SCRIPT THEN PASTE IT IN PIPELINE SCRIPT**

```
STAGE ("OWASP") {
        STEPS {
          DEPENDENCYCHECK ADDITIONALARGUMENTS: '--SCAN ./ --DISABLEYARNAUDIT --
DISABLENODEAUDIT', ODCINSTALLATION: 'DP-CHECK'
          DEPENDENCYCHECKPUBLISHER PATTERN: '**/DEPENDENCY-CHECK-REPORT.XML'
        }
      }
```



STEP-18: GO TO PIPELINE SYNTAX

1. **SELECT (dependencycheckpublisher)**
2. **IN THE XML REPORT PATTERN BOX GIVE (**/DEPENDENCY-CHECK-REPORT.XML)**
3. **COPY THE SCRIPT THEN PASTE IT INPIPELINE SCRIPT**



STEP-19: GO TO PIPELINE JOB THEN NEXT STEP IN PIPELINE IS TRIVY SCAN THE TRIVY IS A TOOL IT IS USED SCAN THE DOCKER IMAGES AND THE BUILD THE DOCKERFILE TO CREATE IMAGE

```
stage ("Trivy scan") {
        steps {
            sh "trivy fs . > trivyfs.txt"
        }
    }
    stage ("Build Dockerfile") {
        steps {
            sh 'docker build -t image1 .'
```

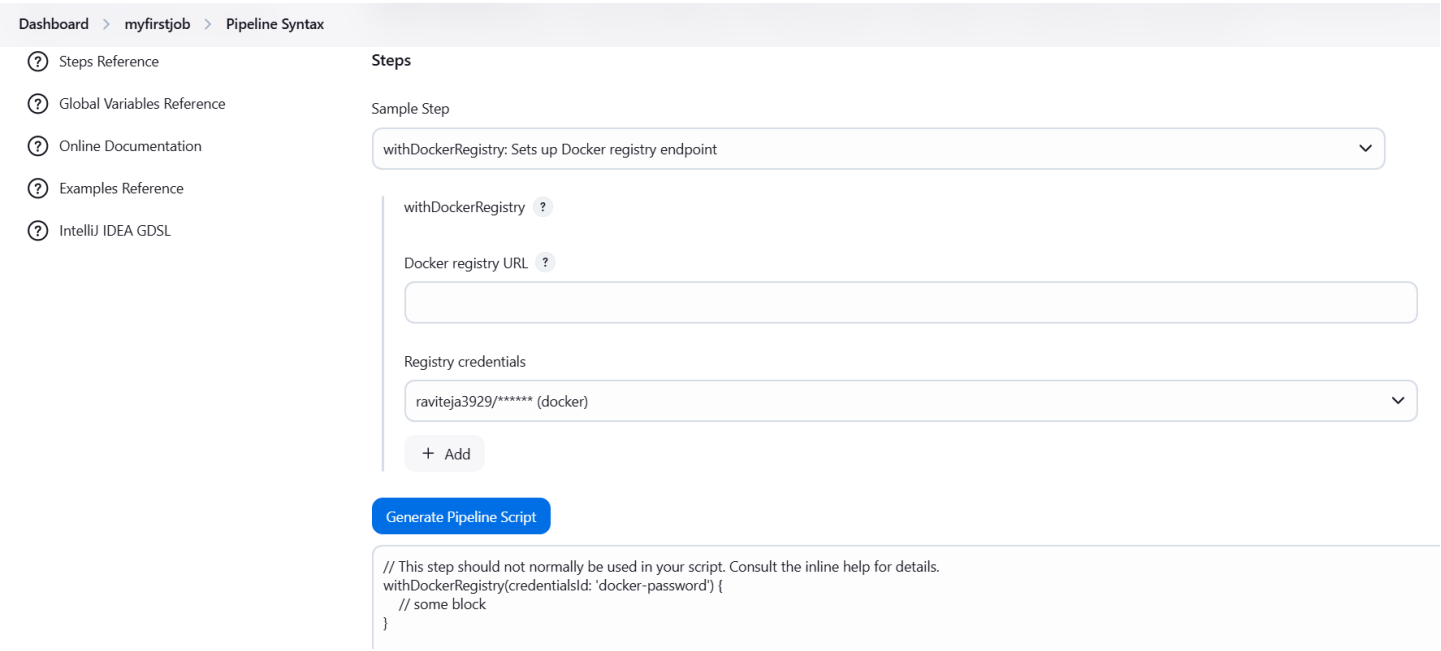
```
    }
}
```

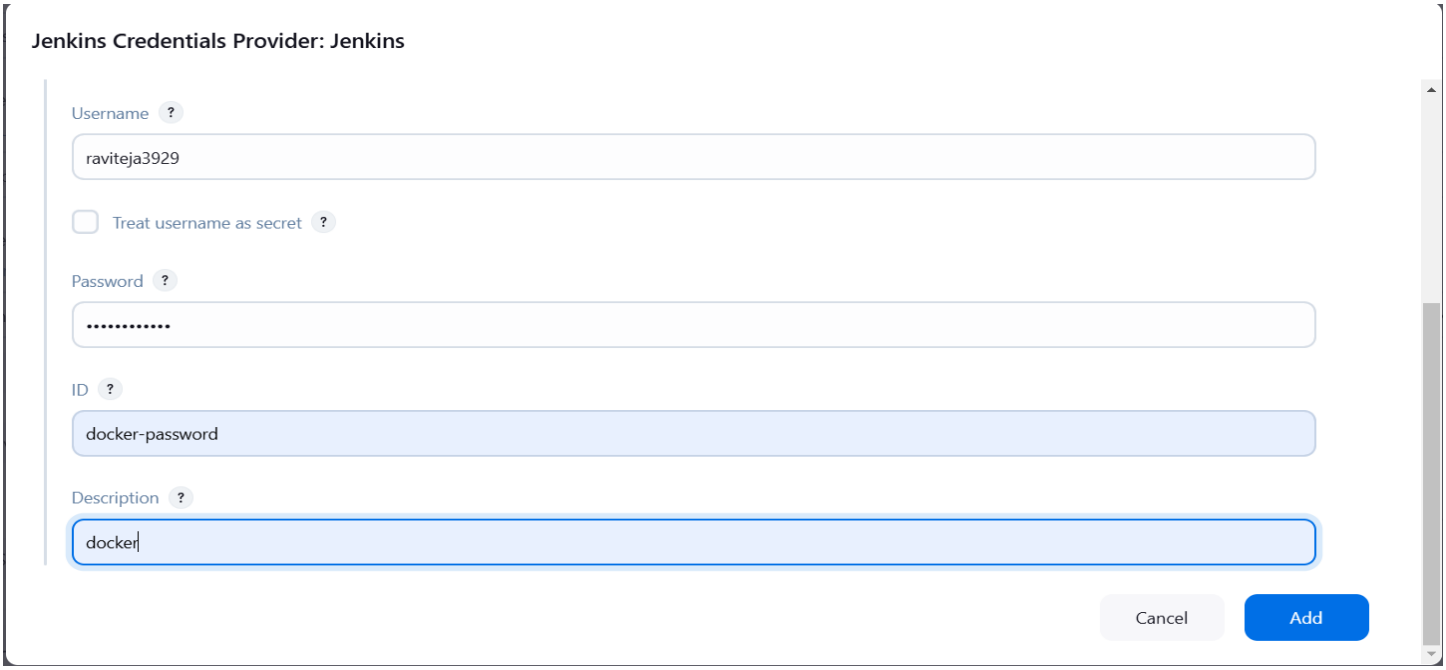
```
45            }
46        }
47        stage ("Trivy scan") {
48            steps {
49                sh "trivy fs . > trivyfs.txt"
50            }
51        }
52
53        stage ("Build Dockerfile") {
54            steps {
55                sh 'docker build -t image1 .'
56            }
57        }
```

STEP-20: GO TO PIPELINE SYNTAX

1. **SELECT (withdockerregistry)**
2. **CLICK ADD REGISTRY CREDENTIALS THEN GIVE DOCKERHUB REGISTRY CREDENTIALS**
3. **IN COPY THE SCRIPT THEN PASTE IT IN PIPELINE SCRIPT**

```
STAGE("DOCKER BUILD & PUSH"){
        STEPS{
          SCRIPT{
            WITHDOCKERREGISTRY(CREDENTIALSID: 'DOCKER-PASSWORD') {
              SH "DOCKER TAG IMAGE1 RAVITEJA3929/LOKI:MYDOCKER"
              SH "DOCKER PUSH RAVITEJA3929/LOKI:MYDOCKER"
            }
          }
        }
      }
```



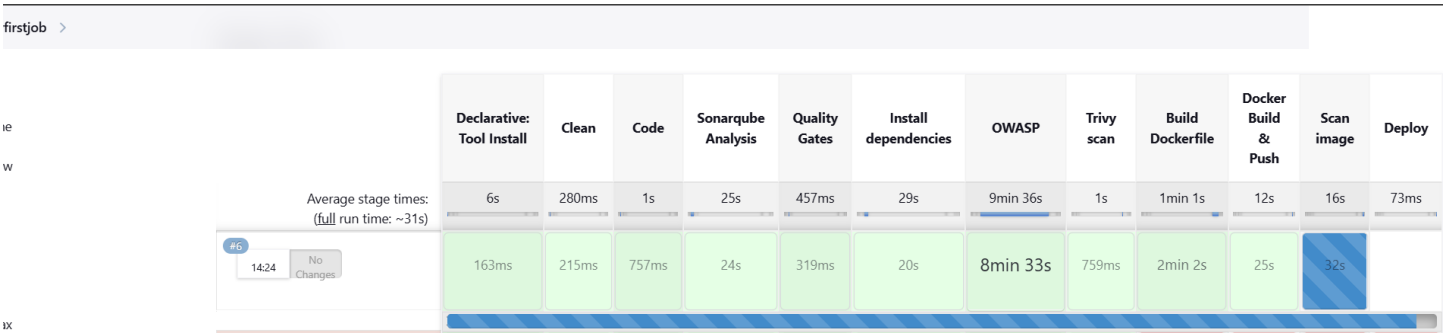


STEP-21: GO TO PIPELINE THEN WRITE NEXT STEPS SCAN IMAGE AND DEPLOY THEN SAVE THE PIPELINE BUILD THE JOB

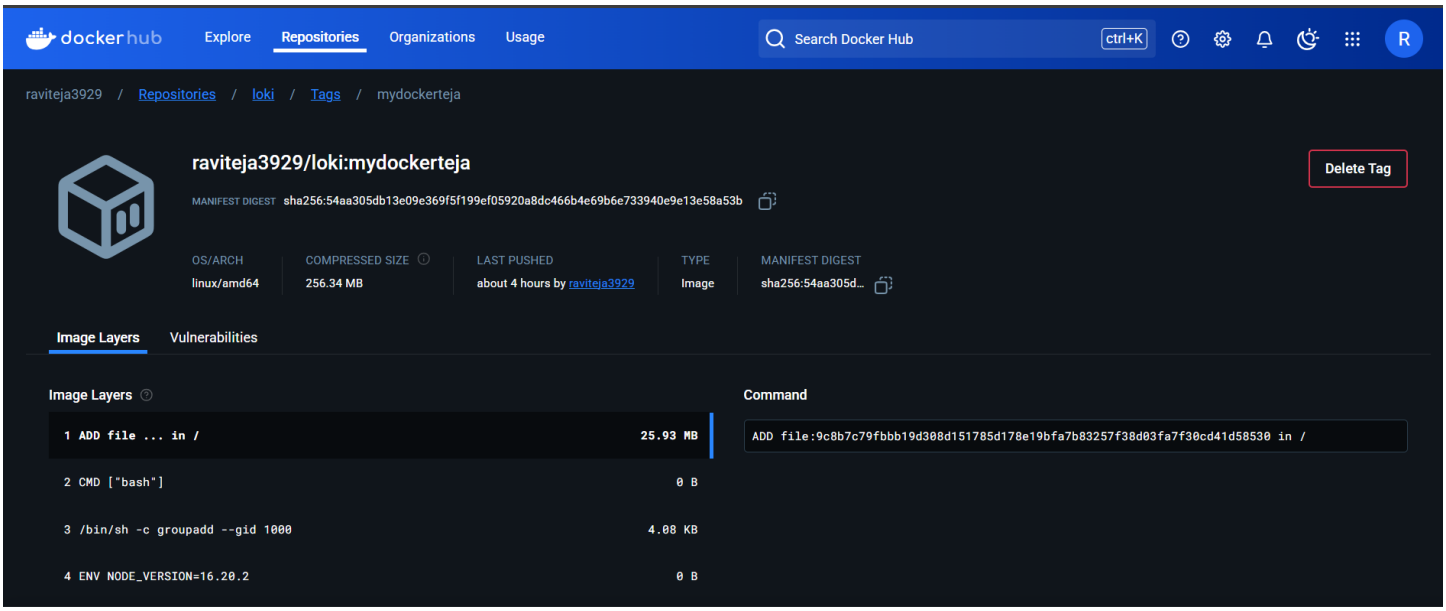
```
        stage ("Scan image") {
            steps {
                sh 'trivy image raviteja3929/loki:mydocker'
            }
        }
        stage ("Deploy") {
            steps {
                sh 'docker run -d --name zomato -p 3000:3000 raviteja3929/loki:mydocker'
            }
        }

    }
}
```

```
stage ("Scan image") {
    steps {
        sh 'trivy image raviteja3929/loki:mydocker'
    }
}
stage ("Deploy") {
    steps {
        sh 'docker run -d --name zomato -p 3000:3000 raviteja3929/loki:mydocker'
    }
}
}
}
```

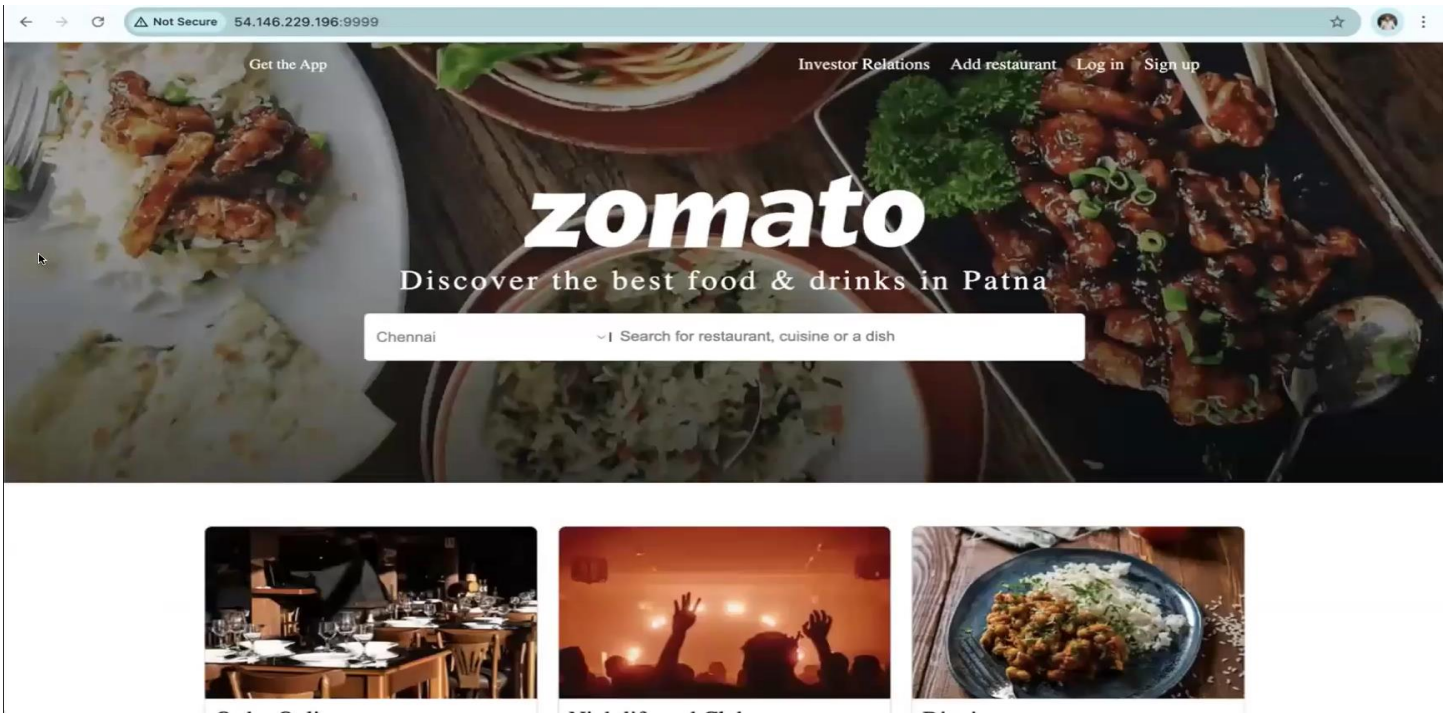
STEP-22: YOU CAN SEE THE PIPELINE GETTING BUILD



STEP-23: GO TO DOCKER REGISTRY AND YOU CAN SEE THE DEPLOY IN THE DOCKER HUB



STEP-24: NOW GO TO NEW TAB COPY AND PASTE PUBLIC-IP ADDRESS AND WITH CONTAINER PORT NUMBER PUBLIC-IP:3000 YOU CAN ACCESS THE DEPLOYMENT



PIPELINE SCRIP :

```
pipeline {
    agent any
    tools {
        jdk 'jdk17'
        nodejs 'node16'
    }
    environment {
        SCANNER_HOME=tool 'mysonar'
    }
    stages {
        stage ("Clean") {
            steps {
                cleanWs()
            }
        }
        stage ("Code") {
            steps {
                git 'https://github.com/RAVITEJA3929/Zomato-Project.git'
            }
        }
        stage("Sonarqube Analysis") {
            steps{
                withSonarQubeEnv('mysonar') {
                    sh ''' $SCANNER_HOME/bin/sonar-scanner -Dsonar.projectName=zomato \
                    -Dsonar.projectKey=zomato '''
                }
```

```
        }
    }
    stage ("Quality Gates") {
        steps {
            script {
                waitForQualityGate abortPipeline: false, credentialsId: 'sonar-token'
            }
        }
    }
    stage ("Install dependencies") {
        steps {
            sh 'npm install'
        }
    }
    stage ("OWASP") {
        steps {
            dependencyCheck additionalArguments: '--scan ./ --disableYarnAudit --disableNodeAudit', odcInstallation: 'Dp-
Check'
            dependencyCheckPublisher pattern: '**/dependency-check-report.xml'
        }
    }
    stage ("Trivy scan") {
        steps {
            sh "trivy fs . > trivyfs.txt"
        }
    }

    stage ("Build Dockerfile") {
        steps {
            sh 'docker build -t image1 .'
        }
    }
```

```
    stage("Docker Build & Push"){
      steps{
        script{
          withDockerRegistry(credentialsId: 'docker-password') {
              sh "docker tag image1 raviteja3929/loki:mydocker"
              sh "docker push raviteja3929/loki:mydocker"
           }
        }
      }
    }
    stage ("Scan image") {
      steps {
        sh 'trivy image raviteja3929/loki:mydocker'
      }
    }
    stage ("Deploy") {
      steps {
        sh 'docker run -d --name zomato -p 3000:3000 raviteja3929/loki:mydocker'
      }
    }
  }
}
```